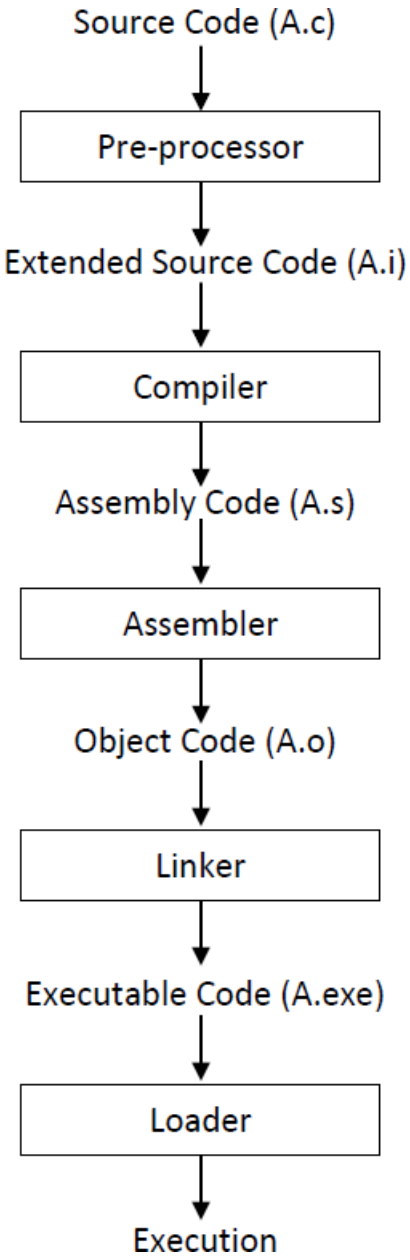
PRACTICAL LIST

CEUC103: Computer Programming Foundation Semester: 1st | Total Labs: 30 | Total Practical Hours: 60

Practical No.	Problem Definition	Hours	CO
1	**Aim**: To understand the C program compilation process by writing a simple **"Hello World"** program, generating intermediate compilation files (.i, .s, .o), creating an executable file, and executing the program in Ubuntu Linux as well as Turbo C and Code::Blocks on Windows. **Industry Scenario** ABC Embedded Systems Pvt. Ltd. develops software for embedded devices using the programming language. Before deploying software on embedded hardware, every program undergoes several stages including preprocessing, compilation, assembly, linking, loading, and execution. As a **Software Engineer Trainee**, you have been assigned the task of understanding the complete compilation process by developing a simple **"Hello World"** program. The management expects you to generate and examine every intermediate file produced during compilation to understand how source code is converted into executable machine code. Based on the above requirements, write a program to print **"Hello World"**, generate all intermediate compilation files, execute the program successfully, and observe each stage of the compilation process. **Tasks to be Performed** 1. Install **Ubuntu 18.04.6 LTS** on Windows or use an online Linux environment. 2. Install the required packages (**gcc**, **cpp**, and **as**) if they are not already available. 3. Create a C source file (**A.c**) using the Linux text editor (**vi**) or Notepad. 4. Write a C program to display **"Hello World"**. 5. Generate the preprocessed source file (**A.i**) using the C Preprocessor. 6. Generate the assembly language file (**A.s**) using the GCC compiler. 7. Generate the object file (**A.o**) using the GNU Assembler. 8. Link the object file to create the executable file (**A.exe**). 9. Execute the generated executable file and verify the output. 10. Compile and execute the same program using **Turbo C** and **Code::Blocks** in Windows.	4	CO1, 2

11. Observe and document the role of each compilation stage from source code to execution.
12. Submit all generated files (**A.c, A.i, A.s, A.o, A.exe**) along with the practical report.

Process of Compiling & Executing a Program



Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Explain the purpose of each stage of the compilation process: preprocessing, compilation, assembly, linking, loading, and execution.
2. Differentiate between source code, assembly code, object code, and executable code.
3. Why is the linker necessary even for a simple "Hello World" program?

4. Compare the compilation process in Ubuntu Linux with the compilation process in Turbo C and Code::Blocks.

Supplementary Problems

1. Write a program to display your name, enrollment number, and branch, and generate all intermediate compilation files.
2. Develop a program to perform the addition of two numbers and observe each compilation stage.
3. Create a program to calculate the area of a rectangle and generate the corresponding .i, .s, .o, and executable files.

Key Skills to be Addressed

- Understanding the program compilation process.
- Working with Linux command-line tools.
- Using GCC, CPP, and GNU Assembler.
- Creating and executing programs in Ubuntu Linux.
- Compiling programs using Turbo C and Code::Blocks.
- Understanding executable file generation.

Learning Outcome

Upon successful completion of this practical, students will be able to write and execute simple programs, explain every stage of the compilation process, generate intermediate compilation files, create executable programs using GCC, and compare program execution across Linux and Windows development environments.

Software / Tools / Platforms to be Used

- Ubuntu 18.04.6 LTS
- GCC Compiler
- C Preprocessor (CPP)
- GNU Assembler (AS)
- Linux Terminal
- Turbo C
- Code::Blocks
- Notepad / vi Editor

Total Hours of Engagement

4 Hours

- **3 Hours:** Program Development, Compilation, and Execution
- **1 Hour:** Observation, Documentation, and Faculty Evaluation

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of the C program, successful generation of .i, .s, .o, and executable files, and correct execution of the program.	**40%**

<table>
<tr><td></td><td><table>
<tr><td>Ability to explain the preprocessing, compilation, assembly, linking, loading, and execution stages of a C program.</td><td>25%</td></tr>
<tr><td>Correct use of Linux commands, GCC tools, and successful execution in Ubuntu Linux, Turbo C, and Code::Blocks.</td><td>20%</td></tr>
<tr><td>Code readability, documentation, observations, file organization, and adherence to programming standards.</td><td>15%</td></tr>
</table></td><td></td><td></td></tr>
<tr><td>2</td><td>Aim: To analyse a real-world computational problem by identifying inputs, outputs, and processes; applying problem decomposition; designing algorithms, flowcharts, pseudocode, and test cases; and developing the initial version of the Student Record Management System.

Industry Scenario: CHARUSAT University currently maintains student records manually using paper files and spreadsheets. As the number of students increases every academic year, maintaining accurate records, searching student information, updating academic details, and generating reports have become time-consuming and error-prone.

To improve efficiency, the university has decided to develop a Student Record Management System (SRMS). Before developing the complete software, the software development team must understand the problem thoroughly by analysing system requirements, identifying inputs and outputs, decomposing the problem into manageable modules, designing the solution, and validating the logic using appropriate test cases.

As a software developer, your responsibility is to analyse the problem and develop the first working prototype of the Student Record Management System, which will be extended in subsequent practical sessions throughout the semester.

Practical 2.1
Tasks to be Performed Part
A: Problem Analysis
Study the given scenario carefully and perform the following activities:

1. Identify the stakeholders of the system.
2. Identify the system inputs.
3. Identify the expected outputs.
4. Identify the major processing steps.
5. Prepare an Input–Process–Output (IPO) Chart.

Part B: Problem Decomposition</td><td>4</td><td>CO1</td></tr>
</table>

Break the Student Record Management System into smaller functional modules.
For example,

- Student Registration
- Student Search
- Student Record Update
- Student Record Deletion
- Attendance Management
- Result Processing
- Report Generation
- Data Storage

Represent the modules using a suitable hierarchy diagram.

Part C: Algorithm Design
Design an algorithm for the Student Registration Module.
The module should show the following information:

- Enrolment Number
- Student Name
- Branch
- Semester
- Mobile Number

Display all the entered information in a properly formatted manner.

Part D: Flowchart Design
Draw a flowchart representing the Student Registration Module.

Example Format (For understanding, not for Implementation)

Test Case	Input	Expected Output
TC01	Valid Enrolment Number	Record Accepted
TC02	Blank Student Name	Error Message
TC03	Semester = 0	Invalid Semester
TC04	Semester = 9	Invalid Semester

Part E: Manual Debugging

Perform a dry run of the algorithm using

- One valid test case
- One invalid test case

Part F: Program Development

Develop a console-based application that performs the following tasks:

1. **Display the title of the application.**

****
* STUDENT RECORD MANAGEMENT SYSTEM
****

2. **Make static output by putting the following student information:**
- Enrollment Number
- Student Name
- Branch
- Semester
- Mobile Number

3. Display the entered student information in a well-formatted manner.

Expected Output

****
* STUDENT RECORD MANAGEMENT SYSTEM
****

Enter Enrollment Number : 26CSE001
Enter Student Name : Akshar Patel Enter
Branch : CSE
Enter Semester : 1
Enter Mobile Number : 9999999999

Student Information

Enrollment Number : 26CSE001
Student Name : Akshar Patel Branch
: CSE
Semester : 1
Mobile Number : 9999999999

Practical 2.2

Tasks to be Performed

Enhance the Student Record Management System developed in Practical 2.1 by performing the following tasks:

1. Display the following project information before accepting student details:
    - Software Version
    - Academic Year
    - Institute Name
2. Accept the following student details:
    - Enrollment Number
    - Student Name
    - Branch
    - Semester
    - Mobile Number
3. Display all the entered student information in a well-formatted report.

4. Execute the program using the selected development environment and observe the program execution.

Expected Output

```
*****************************************
     STUDENT RECORD MANAGEMENT SYSTEM
*****************************************

Software Version : 1.1
Institute        : CHARUSAT University
Academic Year    : 2026-27

-----------------------------------------
Student Registration
-----------------------------------------

Enter Enrollment Number :
Enter Student Name :
Enter Branch :
Enter Semester :
Enter Mobile Number :

-----------------------------------------
Student Information
-----------------------------------------

Enrollment Number :
Student Name      :
Branch            :
Semester          :
Mobile Number     :
-----------------------------------------
```

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Identify the Input, Process, and Output components of the Student Record Management System.
2. Explain how problem decomposition simplifies the development of large software systems.
3. Compare Algorithm, Flowchart, and Pseudocode. Which representation is more suitable during the planning phase and why?
4. Why should software be tested using valid, invalid, and boundary test cases before implementation?
5. Explain how manual tracing helps in identifying logical errors during software development.

Supplementary Problems

1. Extend the registration module to additionally add:
   - Date of Birth
   - Gender
   - Blood Group
2. Display the student information inside a formatted table.
3. Register the information of three students sequentially without using iterative constructs.

Key Skills to be Addressed

- Problem interpretation
- Requirement analysis
- Input–Process–Output analysis
- Problem decomposition
- Algorithm development
- Flowchart preparation
- Pseudocode writing
- Test case design
- Manual debugging
- Console-based program development
- Formatted input and output

Learning Outcome

Upon successful completion of this practical, students will be able to analyse computational problems, identify system inputs, outputs, and processes, decompose complex problems into smaller modules, design algorithms, flowcharts, pseudocode, prepare appropriate test cases, manually validate program logic, and develop the initial version of a Student Record Management System.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler
- Ubuntu Linux

Total Hours of Engagement 4 Hours

- **120 Minutes** – Problem Analysis and Design
- **60 Minutes** – Algorithm, Flowchart, Pseudocode and Test Case Preparation
- **60 Minutes** – Program Development and Testing

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Problem analysis, IPO chart, problem decomposition, algorithm, flowchart, and pseudocode	30
Correctness of program development and output	35
Test case design, manual debugging, and logical validation	20
Code readability, documentation, formatting, and viva performance	15

3	**Aim** To understand programming paradigms, language hierarchy, language processors, compiler phases, program execution, coding standards, error identification, and code optimization by enhancing the Student Record Management System. **Industry Scenario** The first prototype of the **Student Record Management System (SRMS)** has been successfully developed. Before deploying the next version of the software, the development team must establish a standard programming environment, understand how a program is translated into an executable application, and follow coding practices adopted in professional software development. The university administration has now requested the development team to extend the system by incorporating students' academic information. The enhanced system should record marks obtained in different subjects, calculate academic statistics, and support controlled modification of marks during the result verification process. During moderation, marks may be increased or decreased because of grace marks, rechecking, or correction of data-entry mistakes. Therefore, the software should correctly implement arithmetic operators, including **pre-increment, post-increment, pre-decrement, and post-decrement**, while maintaining proper coding standards and software quality. As a software developer, your responsibility is to enhance the existing Student Record Management System by incorporating academic information, understanding operator behavior during mark processing, identifying programming errors, and following standard coding practices. **Practical 3.1** **Tasks to be Performed** Enhance the Student Record Management System developed in Practical 2.2 by performing the following tasks: 1. Extend the application to accept marks of the following subjects: o Mathematics o Physics o Computer Programming Foundation 2. Calculate: o Total Marks o Average Marks o Percentage 3. Display the complete academic summary along with the student information.	4	CO2

4. Compile and execute the application successfully.

Expected Output

```
*****************************************
     STUDENT RECORD MANAGEMENT SYSTEM
*****************************************

Software Version : 1.2

-----------------------------------------
Student Registration
-----------------------------------------

Enter Enrollment Number : 24CE001
Enter Student Name      : Amit Patel
Enter Branch            : CE
Enter Semester          : 1
Enter Mobile Number     : 9876543210

-----------------------------------------
Academic Information
-----------------------------------------

Enter Mathematics Marks             : 82
Enter Physics Marks                 : 76
Enter Programming Foundation Marks  : 91

-----------------------------------------
Academic Summary
-----------------------------------------

Total Marks       : 249
Average Marks     : 83.00
Percentage        : 83.00%

-----------------------------------------
Student Information
-----------------------------------------

Enrollment Number : 24CE001
Student Name      : Amit Patel
Branch            : CE
Semester          : 1
Mobile Number     : 9876543210
-----------------------------------------
```

Practical 3.2

Tasks to be Performed

Enhance the **Student Record Management System** developed in **Practical 3.1** by performing the following tasks:

1. Improve the program by applying:
    - Meaningful variable names
    - Proper indentation
    - Appropriate comments
    - Standard naming conventions
    - Consistent output formatting

2. Assume the following marks are already stored in the Student Record Management System after result processing:
   - Computer Programming Foundation = **70**
3. Execute the following statements independently and observe the output:

```
++cpfMarks;
--cpfMarks;
cpfMarks++;
cpfMarks--;
```

4. Complete the following observation table based on the program output.

Sr. No.	Statement	Marks Before Execution	Marks After Execution
1	++cpfMarks		
2	--cpfMarks		
3	cpfMarks++		
4	cpfMarks--		

5. Assume the following marks are available in the Student Record Management System:
   - Computer Programming Foundation = **70**
   - Mathematics = **80**
6. Evaluate the following expression:

```
result = ++cpfMarks + cpfMarks++ + --mathMarks + ++mathMarks - mathMarks--;
```

7. Complete the following observation table.

Subject	Marks Before Evaluation	Marks After Evaluation
Computer Programming Foundation		
Mathematics		

8. Determine the value of **result** and explain its evaluation **step-by-step** by showing how the values stored in memory change after each operation.
9. Identify and rectify the following categories of programming errors using the developed application:
   - Lexical Errors

- Syntax Errors
- Semantic Errors
- Runtime Errors
- Logical Errors

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Differentiate between Procedural, Object-Oriented, Meta, and Reflective programming paradigms.
2. Compare Machine Language, Assembly Language, and High-Level Language.
3. Explain the role of the Editor, Compiler, Debugger, and IDE during software development.
4. Describe the functions of the Preprocessor, Compiler, Assembler, Linker, and Loader in converting a source program into an executable program.
5. Explain the different phases of a compiler.
6. Differentiate between lexical, syntax, semantic, runtime, and logical errors with suitable examples.
7. Why are meaningful identifiers, indentation, comments, and naming conventions important in program development?
8. Explain the importance of code optimization in software development.

Supplementary Problems

1. Modify the program to accept marks of **five** subjects and calculate the Total, Average, and Percentage.
2. Display the student's academic summary in a tabular format with proper alignment.
3. Prepare an observation chart showing the complete program translation process from source code to executable program.

Key Skills to be Addressed

- Programming environment setup
- Program compilation and execution
- Academic information processing
- Arithmetic expression implementation
- Program documentation
- Coding standards
- Error identification and debugging
- Code optimization
- Formatted input and output

Learning Outcome

Upon successful completion of this practical set, students will be able to execute programs using a standard programming environment, understand the software translation and execution process, process academic information using arithmetic expressions, identify and
rectify programming errors, apply coding standards, and enhance program readability and maintainability.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler
- Ubuntu Linux (Optional)

Total Hours of Engagement 4 Hours

- **120 Minutes** – Enhancement of Student Record Management System
- **60 Minutes** – Academic Information Processing and Program Execution
- **60 Minutes** – Error Analysis, Coding Standards, and Code Optimization
-

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Enhancement of the Student Record Management System and correct implementation of academic information processing	35
Understanding of program execution, language processors, compiler phases, and programming tools	25
Error identification, coding standards, and code optimization	25
Program execution, documentation, output formatting, and viva performance	15

4 | **Decision Making in Student Record Management System** | 6 | CO3

Aim

To enhance the Student Record Management System by implementing decision-making constructs for evaluating student performance and developing a menu-driven application.

Industry Scenario

The Student Record Management System developed in the previous practicals is capable of registering student information and calculating academic statistics. However, the Examination Cell still performs important academic decisions manually, such as determining pass/fail status, assigning grades, and generating academic remarks.

Additionally, the current application executes sequentially, requiring users to rerun the program for every operation. To

improve usability, the software should provide a menu-driven interface that allows administrators to perform different operations through a single application.

As a software developer, your responsibility is to enhance the Student Record Management System by implementing decision-making constructs and developing an interactive menu-driven application.

Practical 4.1

Topics Covered

- Conditional Statements
  - Simple if
  - if-else

Tasks to be Performed

Enhance the Student Record Management System developed in **Practical 3** by implementing the following features.

1. Determine whether the student has **PASSED** or **FAILED** based on the calculated percentage.
2. Display the student's academic result along with the previously generated academic summary.
3. Display an appropriate message indicating the student's academic status.

Expected Output

```
****************************************
     STUDENT RECORD MANAGEMENT SYSTEM
****************************************

----------------------------------------
Academic Summary
----------------------------------------

Total Marks       : 249
Average Marks     : 83.00
Percentage        : 83.00%

----------------------------------------
Academic Result
----------------------------------------

Result            : PASS

Congratulations! You have successfully passed.

----------------------------------------
```

Practical 4.2

Topics Covered

- else-if Ladder

Tasks to be Performed

Enhance the Student Record Management System developed in Practical 4.1 by implementing the following features.

1. Determine the student's **Grade** based on the calculated percentage.
2. Display an appropriate **Performance Remark** corresponding to the assigned grade.
3. Display the complete academic report including:
   - Total Marks
   - Average Marks
   - Percentage
   - Result
   - Grade
   - Performance Remark

Suggested Grading Policy

Percentage	Grade	Performance Remark
90 – 100	O	Outstanding
80 – 89	A+	Excellent
70 – 79	A	Very Good
60 – 69	B+	Good
50 – 59	B	Satisfactory
40 – 49	C	Needs Improvement
Below 40	F	Failed

Expected Output

```
****************************************
     STUDENT RECORD MANAGEMENT SYSTEM
****************************************

----------------------------------------
Academic Summary
----------------------------------------

Total Marks       : 249
Average Marks     : 83.00
Percentage        : 83.00%

----------------------------------------
Academic Result
----------------------------------------

Result            : PASS
Grade             : A+
Performance       : Excellent

----------------------------------------
```

Practical 4.3

Topics Covered

- switch-case

Tasks to be Performed

Transform the Student Record Management System developed in Practical 4.2 into a **menu-driven application**.

Implement the following menu options using the **switch-case** statement.

Menu Options

1. Register New Student
2. Display Student Record
3. Enter Student Marks
4. Display Academic Result
5. Exit

Functional Requirements

Option 1 – Register New Student

Accept the following details:

- Enrollment Number
- Student Name
- Branch
- Semester
- Mobile Number

	Display: **Student Registered Successfully.** *Option 2 – Display Student Record* **(Use Goto)** Display all student information entered through Option 1. *Option 3 – Enter Student Marks* **(Use Goto)** Accept marks for the following subjects: • Mathematics • Physics • Computer Programming Foundation Store the entered marks for generating the academic result. Display: **Marks Entered Successfully.** *Option 4 – Display Academic Result* **(Use Goto)** Using the marks entered through Option 3, display: • Total Marks • Average Marks • Percentage • Result • Grade • Performance Remark *Option 5 – Exit* Terminate the application.		

Expected Output

```
*****************************************
     STUDENT RECORD MANAGEMENT SYSTEM
*****************************************

-------------- MAIN MENU -------------------

1. Register New Student
2. Display Student Record
3. Enter Student Marks
4. Display Academic Result
5. Exit

Enter Your Choice : 1

-----------------------------------------
Student Registration
-----------------------------------------

Enrollment Number : 24CE001
Student Name      : Amit Patel
Branch            : CE
Semester          : 1
Mobile Number     : 9876543210

Student Registered Successfully.

-----------------------------------------
```

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Differentiate between if, if-else, else-if ladder, and switch-case statements.
2. Why is the if-else statement suitable for determining PASS or FAIL?
3. Why is the else-if ladder more appropriate than multiple if statements for grade generation?
4. In which situations is the switch-case statement preferred over an else-if ladder?
5. Explain how the menu-driven Student Record Management System improves the usability of the application.
6. How do decision-making constructs help automate academic evaluation in the Student Record Management System?

Supplementary Problems

1. Extend the grading policy to include **Distinction** and **First Class** classifications.

2. Modify the menu to include an option for displaying the grading policy.
3. Add a confirmation message before exiting the application.
4. Display the student's eligibility for a merit certificate based on the assigned grade.

Key Skills to be Addressed

- Decision-making using conditional statements
- Multi-way selection using switch-case
- Menu-driven application development
- Academic result generation
- Grade evaluation
- Performance analysis
- Incremental enhancement of an existing software application

Learning Outcome

Upon successful completion of this practical set, students will be able to implement decision-making constructs using if, if-else, else-if ladder, and switch-case statements to automate academic evaluation and develop a menu-driven Student Record Management System.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler

Total Hours of Engagement 6 Hours

- **50 Minutes** – Result Evaluation using if and if-else
- **70 Minutes** – Grade Generation using else-if Ladder
- **120 Minutes** – Menu-Driven Application using switch-case
- **120 Minutes** – Extra Problem Solving based on similar concept

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of conditional statements and switch-case	35
Enhancement of the Student Record Management System with result generation and menu-driven functionality	30
Program correctness, testing, and formatted output	20

	Code readability, documentation, and viva performance	15		

5	**Iterative Processing in Student Record Management System** **Aim** To enhance the Student Record Management System by implementing iterative constructs for repetitive data entry, batch processing of student records, and continuous execution of menu-driven operations. **Industry Scenario** The Student Record Management System developed in the previous practicals successfully registers student details, records academic marks, evaluates student performance, and provides a menu-driven interface. During pilot deployment, the Examination Cell identified the following operational limitations: • The application accepts marks for only three predefined subjects, whereas different academic programs may have different numbers of subjects. • The application processes only one student's record at a time, requiring the operator to restart the software for each new student. • The Main Menu executes only once and terminates after a single operation, reducing the usability of the system. To improve flexibility and efficiency, the software development team has been assigned to enhance the Student Record Management System using iterative constructs. **Practical 5.1** Topics Covered • for Loop **Tasks to be Performed** Enhance the Student Record Management System developed in Practical Set 4 by implementing the following features. 1. Modify **Option 3 – Enter Student Marks** to accept the **number of subjects** offered to the student. 2. Accept marks of all subjects using a **for loop**. 3. Calculate and display: o Total Marks o Average Marks o Percentage o Result o Grade o Performance Remark 4. Verify the academic report for different numbers of subjects.	6	CO3

Expected Output

```
*****************************************
     STUDENT RECORD MANAGEMENT SYSTEM
*****************************************

Enter Number of Subjects : 5

Enter Marks of Subject 1 : 85
Enter Marks of Subject 2 : 78
Enter Marks of Subject 3 : 91
Enter Marks of Subject 4 : 82
Enter Marks of Subject 5 : 88

-----------------------------------------
Academic Result
-----------------------------------------

Total Marks       : 424
Average Marks     : 84.80
Percentage        : 84.80%

Result            : PASS
Grade             : A+
Performance       : Excellent

-----------------------------------------
```

Practical 5.2

Topics Covered

- while Loop and do..while

Tasks to be Performed

Enhance the Student Record Management System developed in Practical 5.1 by implementing the following features.

1. Modify **Option 1 – Register New Student** to support **batch student registration**.
2. After registering one student, display the student's information and ask the administrator whether another student's record should be entered.
3. Continue the registration process using a **while loop** until the administrator chooses to stop.
4. Return to the Main Menu after completing the registration session.

Note: At this stage, the application processes one student record at a time. Permanent storage of multiple student records will be implemented in a subsequent practical using arrays.

Expected Output:

```
*****************************************
     STUDENT RECORD MANAGEMENT SYSTEM
*****************************************

Student Registration

Enrollment Number : 24CE001
Student Name      : Amit Patel
Branch            : CE
Semester          : 1
Mobile Number     : 9876543210

Student Registered Successfully.

Register Another Student? (Y/N) : Y

-----------------------------------------

Enrollment Number : 24CE002
Student Name      : Riya Shah
Branch            : CE
Semester          : 1
Mobile Number     : 9876501234

Student Registered Successfully.

Register Another Student? (Y/N) : N

Returning to Main Menu...
```

Practical 5.3

Industry Scenario:

A digital display board manufacturer requires software to generate different character patterns for testing LED display panels. Before developing complex display applications, the software engineer must verify that the display patterns are generated correctly using iterative constructs.

Tasks to be Performed

Develop an application to generate the following patterns using nested loops:

1 – Numeric Pattern	**2 – Lowercase Alphabet Pattern**
1 1 2 1 2 3 1 2 3 4 1 2 3 4 5	a a b a b c a b c d a b c d e
3 – Uppercase Alphabet Pattern	**4 – Right Half Number Triangle**
A A B A B C A B C D A B C D E	1 1 2 1 2 3 1 2 3 4 1 2 3 4 5

5 – Full Number Pyramid	6 – Full Alphabet Pyramid
1 1 2 1 1 2 3 2 1 1 2 3 4 3 2 1 1 2 3 4 5 4 3 2 1	A A B A A B C B A A B C D C B A A B C D E D C B A

Expected Output:

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Why is a **for loop** preferred when the number of repetitions is known beforehand, as in subject-wise mark entry?
2. Why is a **while loop** more suitable than a **for loop** for batch student registration?
3. Why must the Main Menu execute at least once, making the **do-while loop** appropriate for a menu-driven application?
4. What would happen if the Main Menu in Practical 5.3 were implemented using a **while loop** instead of a **do-while loop**?
5. Differentiate between **for**, **while**, and **do-while** loops with suitable applications in the Student Record Management System.
6. Explain how iterative constructs improve the usability and efficiency of menu-driven software applications.

Supplementary Problems

1. Develop a program to accept marks of **N employees** in a training assessment using a **for loop** and calculate the average training score.
2. Develop a menu-driven application using a **do-while loop** to perform the following operations:
    - Find the square of a number
    - Find the cube of a number
    - Check whether a number is even or odd
    - Exit
3. Develop a program to repeatedly accept positive integers using a **while loop** and display:
    - Count of numbers entered
    - Sum of all numbers
    - Average of the entered numbers
4. Develop a menu-driven **Library Management System** prototype that supports the following operations:
    - Add Book Details
    - Display Book Details
    - Issue Book
    - Return Book
    - Exit

Key Skills to be Addressed

- Counter-controlled repetition
- Condition-controlled repetition

- Exit-controlled repetition
- Menu-driven application development
- Dynamic academic data processing
- Batch processing
- Incremental software enhancement

Learning Outcome

Upon successful completion of this practical set, students will be able to implement for, while, and do-while loops to automate repetitive tasks, process variable academic data, perform batch student registration, and develop a continuously executing menu-driven Student Record Management System.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler
- Ubuntu Linux (Optional)

Total Hours of Engagement

6 Hours

- **120 Minutes** – Dynamic Subject-wise Mark Entry using for Loop
- **60 Minutes** – Batch Student Registration using while Loop
- **60 Minutes** – Continuous Menu Execution using do-while Loop

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of for, while, and do-while loops	35
Enhancement of the Student Record Management System with iterative processing	30
Program correctness, testing, and formatted output	20
Code readability, documentation, and viva performance	15

6	**Sequential Data Collections (Arrays)** **Aim** To implement one-dimensional arrays for storing, traversing, searching, sorting, and manipulating multiple records, and to apply these concepts in the Student Record Management System. **Practical 6.1** **Industry Scenario** The Sports Committee of CHARUSAT is organizing an inter-department athletics event. The event coordinator wants a software application to record participants' scores and generate a performance summary. Since multiple participants compete in the event, the application must efficiently store and process several records using sequential data collections. As a software developer, your responsibility is to develop a **Sports Event Score Analysis System** using one-dimensional arrays. **Tasks to be Performed** Develop a **Sports Event Score Analysis System** that performs the following tasks: Accept the number of participants. Store the following information using one-dimensional arrays: Participant ID Participant Name Event Score Display all participant records in tabular format. Calculate and display: Total Score Average Score Highest Score Lowest Score ********** SPORTS EVENT SCORE ANALYSIS ********** Enter Number of Participants: 5 Enter Participant ID: P101 Enter Participant Name: Aarav Enter Score: 89 Enter Participant ID: P102 Enter Participant Name: Riya Enter Score: 76 --- Participant Performance --- ID Name Score --- P101 Aarav 89 P102 Riya 76 P103 Karan 91 P104 Neha 82	4	CO3

```
P105    Meera          87
-------------------------------------------

Total Score: 425
Average Score: 85.00
Highest Score: 91
Lowest Score: 76
```

Practical 6.2

Industry Scenario

After successfully recording participants' scores, the Sports Committee wants to quickly locate a participant's performance and prepare the final ranking list based on scores.

As a software developer, enhance the Sports Event Score Analysis System to support searching and sorting operations.

Tasks to be Performed:

Enhance the **Sports Event Score Analysis System** developed in Practical **6.1** by implementing the following features:

Search a participant using the Participant ID.

Display the participant's complete details.

Sort participant records in descending order of scores.

Display the ranking list after sorting.

Display the Top Three performers.

Expected Output

```
**********************************************
        SPORTS EVENT SCORE ANALYSIS
**********************************************

 Search Participant
Enter Participant ID: P103

---------------------------------
        Participant Found
---------------------------------
 ID        : P103
Name    : Karan
Score    : 91

 ---------------------------------
             Ranking List
 ---------------------------------

  Rank  Name        Score
   1      Karan        91
   2      Aarav        89
   3      Meera        87
   4      Neha         82
   5      Riya         76
---------------------------------

   Top Three Performers
   1. Karan   - 91
   2. Aarav   - 89
```

3. Meera - 87

Practical 6.3

The Student Record Management System currently processes only one student's information. The university administration now requires the software to maintain records for an entire class. During the semester, new admissions are confirmed, examination results are revised, and some students cancel their admissions. Therefore, the software should support insertion, updation, and deletion of student records stored in memory.

As a software developer, enhance the Student Record Management System by implementing array manipulation operations without using any external database or file.

Tasks to be Performed

Enhance the **Student Record Management System** developed in **Practical Set 5** by implementing the following features:

- Accept the number of students.
- Store the following information using arrays:
    1. Enrollment Number
    2. Student Name
    3. Percentage
    4. Grade
- Insert a new student record at a specified position.
- Update the percentage and grade of an existing student using the Enrollment Number.
- Delete a student record from a specified position by shifting the remaining records.
- Display the updated student records after every operation.

Expected Output

```
*********************************************
  STUDENT RECORD MANAGEMENT SYSTEM
*********************************************

Enter Number of Students : 3

Enter Student Details

25CS001   Amit Patel    86.40   A+
25CS002   Riya Shah     79.20   A
25CS003   Karan Patel   91.60   O

-----------------------------------------
Current Student Records

25CS001   Amit Patel    86.40
25CS002   Riya Shah     79.20
25CS003   Karan Patel   91.60

-----------------------------------------

Insert New Student
```

Enter Position : 2

Enter Enrollment Number : 25CS004
Enter Student Name : Priya Shah
Enter Percentage : 88.60
Enter Grade : A+

Record Inserted Successfully.

Updated Student Records

25CS001 Amit Patel 86.40
25CS004 Priya Shah 88.60
25CS002 Riya Shah 79.20
25CS003 Karan Patel 91.60

Update Student Record

Enter Enrollment Number : 25CS002

Enter New Percentage : 82.50
Enter New Grade : A+

Record Updated Successfully.

Delete Student Record

Enter Position : 1

Record Deleted Successfully.

Final Student Records

25CS004 Priya Shah 88.60 A+
25CS002 Riya Shah 82.50 A+
25CS003 Karan Patel 91.60 O

Practical 6.4
Industry Scenario
A university department maintains students' internal assessment and end-semester marks in separate tabular forms. Before generating consolidated reports, the Examination Cell wants to understand how tabular data can be processed using two-dimensional arrays. As a software developer, you are assigned to implement matrix operations for processing tabular data efficiently.

Tasks to be Performed

Develop an application to perform the following operations using two-dimensional arrays.

Part A – Matrix Multiplication

- Accept two matrices of order **3 × 3**.
- Perform matrix multiplication.
- Display:
  - First Matrix
  - Second Matrix
  - Resultant Matrix

Part B – Merge Sorted Arrays

- Accept two sorted one-dimensional arrays.
- Merge both arrays into a third array.
- Ensure that the third array remains in **sorted order**.
- Display all three arrays.

Expected Output:

Part A – Matrix Multiplication

```
*****************************************
      MATRIX MULTIPLICATION
*****************************************

First Matrix

2  5  8
3  6  9
4  7  10

Second Matrix

2  3  4
9  7  6
1  5  2

----------------------------------------

Resultant Matrix
57  81  54
69  96  66
81  111  78

----------------------------------------
```

Part B – Merge Two Sorted Arrays

```
*****************************************
    MERGE SORTED ARRAYS
*****************************************

First Array

10 20 30 40 60
```

Second Array

15 25 35 45 55

Merged Sorted Array

10 15 20 25 30 35 40 45 55 60

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Differentiate between linear data organization and tabular data organization.
2. Why are arrays preferred for storing multiple records of the same data type?
3. Explain the purpose of array traversal with suitable examples.
4. Differentiate between searching and sorting operations on arrays.
5. Explain how insertion and deletion are performed in an array. Why is element shifting required?
6. Compare linear search with binary search. Why is linear search more suitable in this practical?
7. What are the advantages and limitations of fixed-size arrays?

Supplementary Problems:

1. Write a program to find the largest number in an integer array.
2. Read N values and print them in reverse order.
3. Calculate the total sum of all numbers stored in an array. [1, 2, 3, 4, 5]
4. Write a program to find the maximum and minimum elements in an array.

Key Skills to be Addressed

- One-dimensional array implementation
- Sequential data storage
- Array traversal
- Linear searching
- Sorting techniques
- Array manipulation
- Record insertion
- Record updation
- Record deletion
- Tabular report generation

Learning Outcome

Upon successful completion of this practical set, students will be able to implement one-dimensional arrays for storing and processing multiple records, perform traversal, searching, sorting, insertion, updation, and deletion operations on arrays, and integrate sequential data collection concepts into the Student

Record Management System for efficient record management.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler

Total Hours of Engagement
4 Hours

- **60 Minutes** – Array Declaration, Traversal, and Aggregate Operations
- **30 Minutes** – Searching and Sorting Using Arrays
- **30 Minutes** – Array Manipulation in Student Record Management System
- **120 Minutes-** matrix operation and merging array

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of array declaration, traversal, searching, sorting, insertion, updation, and deletion	40
Understanding of sequential data collections and array manipulation	25
Correct enhancement of the Student Record Management System	20
Code readability, output formatting, testing, and viva performance	15

7	**String Processing and Pattern Matching** **Aim** To implement string processing techniques using character arrays and standard string functions for concatenation, extraction, pattern matching, and text manipulation, and integrate these operations into the Student Record Management System. **Practical 7.1** **Industry Scenario** The Student Record Management System currently stores student information but does not support textual data processing. The university administration requires additional features to search students by name, generate student references, and process textual information for report generation. As a software developer, enhance the Student Record Management System by implementing string processing operations. **Tasks to be Performed** Enhance the Student Record Management System developed in Practical Set 6 by implementing the following features. 1. Accept the following information: o Enrollment Number o Student Name o Branch	4	CO3

2. Generate a **Student Reference ID** by concatenating the Student Name and Enrollment Number.
3. Extract and display:
    - First Name
    - Last Name
4. Accept a keyword and search whether it exists in the Student Name.
5. Display the complete student report.

Note: Implement the required operations manually wherever possible and then repeat them using appropriate standard string functions.

Expected Output

```
**********************************************
   STUDENT RECORD MANAGEMENT SYSTEM
**********************************************

Enter Enrollment Number : 24CE001

Enter Student Name      : Amit Patel

Enter Branch            : CE

------------------------------------------

Student Reference

Amit Patel-24CE001

------------------------------------------

First Name

Amit

Last Name

Patel

------------------------------------------

Enter Keyword : Patel

Keyword Found.

------------------------------------------

Student Report

Enrollment Number : 24CE001

Student Name      : Amit Patel

Branch            : CE
```

Practical 7.2
Industry Scenario
A publishing company receives articles from different authors for editing before publication. The editorial team wants software to analyse the entered document, perform various string manipulation operations, search for keywords, and generate a formatted report.

As a software developer, develop a **Document Text Analyzer** that performs comprehensive text processing using character arrays and standard string functions.

Tasks to be Performed

Develop a **Document Text Analyzer** that performs the following operations.

Part A – Text Analysis

1. Accept a paragraph of text from the user.
2. Display:

- Total Number of Characters
- Total Number of Words
- Total Number of Alphabets
- Total Number of Digits
- Total Number of Special Characters

Part B – String Extraction

3. Extract and display:

- First Word
- Last Word
- First Character
- Last Character

Part C – String Concatenation

4. Accept another sentence.
5. Concatenate it with the existing paragraph.
6. Display the updated paragraph.

Part D – Pattern Matching

7. Accept a search keyword.
8. Search the keyword in the paragraph.
9. Display whether the keyword is found or not.
10. Display the position of its first occurrence

Part E – String Transformation

11. Convert the paragraph to:

- Uppercase
- Lowercase

12. Reverse the entered paragraph.
13. Accept a word and check whether it is a palindrome

Part F – Standard String Functions

Implement the following operations using standard string library functions.

- Find String Length
- Copy String
- Compare Two Strings
- Concatenate Two Strings
- Search a Substring
- Convert Characters to Uppercase
- Convert Characters to Lowercase

Note: Implement the required operations manually wherever possible. Subsequently, repeat the same operations using appropriate standard string library functions to compare the implementation approaches and generated results.

Expected Output

```
**********************************************
       DOCUMENT TEXT ANALYZER
**********************************************

Enter Paragraph

Programming makes problem solving easier.

---------------------------------------------
Text Analysis
---------------------------------------------

Characters        : 40
Words             : 5
Alphabets         : 35
Digits            : 0
Special Characters : 1

---------------------------------------------
String Extraction
---------------------------------------------

First Word        : Programming
Last Word         : easier
First Character   : P
Last Character    : .

---------------------------------------------
```

```
Concatenation
----------------------------------------

Enter Additional Sentence

Keep learning every day.

Updated Paragraph

Programming makes problem solving easier.
Keep learning every day.

----------------------------------------
Pattern Matching
----------------------------------------

Enter Search Word : learning

Keyword Found

First Occurrence Position : 41

----------------------------------------
String Transformation
----------------------------------------

Uppercase

PROGRAMMING MAKES PROBLEM SOLVING EASIER.
KEEP LEARNING EVERY DAY.

----------------------------------------

Lowercase

programming makes problem solving easier.
keep learning every day.

----------------------------------------

Reverse

.yad yreve gninrael peeK
.reisae gnivlos melborp sekam gnimmargorP

----------------------------------------

Palindrome Check

Enter Word : level

Result : Palindrome

----------------------------------------
Standard String Functions
----------------------------------------
```

Length : 67

String Copied Successfully

Comparison Result : Equal

Concatenation : Successful

Substring Search : Found

toupper() : Successful

tolower() : Successful

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. Differentiate between string data and numerical data with suitable examples.
2. Explain how a string is represented using a character array.
3. Differentiate between manual string manipulation and standard string library functions.
4. Explain the working of string concatenation and string extraction.
5. How does pattern matching help in searching textual information?
6. Differentiate between strcmp() and strstr() functions.
7. Explain the purpose of toupper() and tolower() functions with suitable examples.
8. Differentiate between strlen(), strcpy(), and strcat() functions.

Supplementary Problems

Level 1 – Easy
Develop an application to **count the frequency of vowels and consonants** in a given string.

Level 2 – Medium
Develop an application to **remove all extra spaces** from a given sentence.

Level 2 – Medium
Develop an application to **replace all occurrences of a specified word** with another word in a paragraph.

Level 3 – Hard
Develop an application to **find and display the longest and shortest words** in a given paragraph.

Key Skills to be Addressed

- Character array manipulation
- String processing

- String concatenation
- String extraction
- Pattern matching
- Character classification
- Text transformation
- Standard string library functions
- Text analysis

Learning Outcome

Upon successful completion of this practical set, students will be able to represent textual data using character arrays, perform string concatenation, extraction, pattern matching, text transformation, and character classification, implement both manual and library-based string operations, and integrate string processing techniques into the Student Record Management System.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler

Total Hours of Engagement

4 Hours

120 Minutes – Customer Feedback Analyzer
60 Minutes – String Processing in Student Record Management System
60 Minutes – Document Text Analyzer and Standard String Functions

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of string manipulation, extraction, pattern matching, and standard string functions	40
Enhancement of the Student Record Management System using string processing	25
Program correctness, testing, and output formatting	20
Code readability, documentation, and viva performance	15

8	**Programming Modularization** **Aim** To develop modular applications using user-defined and built-in functions, understand parameter passing mechanisms, implement recursive solutions, and enhance program readability, reusability, and maintainability. **Practical 8.1** **Industry Scenario** The Student Record Management System developed in previous practicals has gradually grown into a large application containing multiple functionalities such as student registration, academic evaluation, grade calculation, and report generation. Maintaining all these operations inside a single program has made the application difficult to understand, modify, and debug. The software development team has decided to modularize the application by dividing it into independent reusable functions. This approach improves program readability, simplifies maintenance, and promotes code reusability. As a software developer, redesign the existing Student Record Management System using appropriate user-defined functions. **Tasks to be Performed** Enhance the **Student Record Management System** developed in **Practical Set 7** by implementing modular programming. Create the following user-defined functions: **displayHeader()** • Display the title of the application. **acceptStudentDetails()** Accept student information including: • Enrollment Number • Student Name • Branch • Semester • Marks **calculateAcademicResult()** • Calculate: • Total Marks • Percentage • Grade	6	CO4

- Result

displayStudentReport()

- Display the complete academic report.
- Call the above functions appropriately from the **main()** function.
- Use appropriate built-in functions wherever applicable for displaying or processing textual information.

Expected Output

```
**********************************************
    STUDENT RECORD MANAGEMENT SYSTEM
**********************************************

---------- Student Registration ----------

Enter Enrollment Number : 24CE001
Enter Student Name      : Amit Patel
Enter Branch            : CE
Enter Semester          : 1

Enter Marks
Subject 1 : 85
Subject 2 : 79
Subject 3 : 88
Subject 4 : 91
Subject 5 : 83

-----------------------------------------

Academic Report

-----------------------------------------

Enrollment Number : 24CE001

Student Name      : Amit Patel

Branch            : CE

Semester          : 1

Total Marks       : 426

Percentage        : 85.20%

Grade             : A+

Result            : PASS

-----------------------------------------
Program Executed Successfully Using User-defined Functions.

-----------------------------------------
```

Practical 8.2
Industry Scenario

A software development organization is preparing coding standards for newly recruited programmers. Before assigning developers to large software projects, the organization wants them to understand different function designs, parameter passing mechanisms, variable scope, and reusable programming techniques.

As a software developer, develop a **Function Design and Parameter Passing Analyzer** to demonstrate different types of user-defined functions and their behaviour.

Tasks to be Performed

Develop a **Function Design and Parameter Passing Analyzer** that demonstrates the following concepts.

Part A – Function Design
Implement the following user-defined functions.

- Function without Parameters and without Return Value.
- Function without Parameters and with Return Value.
- Function with Parameters and without Return Value.
- Function with Parameters and with Return Value.

Display the output generated by each function.

Part B – Parameter Passing

- Implement the following operations.
- Demonstrate **Call by Value** by modifying the value of a variable inside a function and observing the original value after function execution.
- Demonstrate **Call by Reference** by modifying the same variable and observing the changes in the calling function.

Compare both outputs.

Part C – Scope and Visibility

- Declare Local Variables.
- Declare Global Variables.
- Display their values inside and outside user-defined functions.
- Observe the scope and visibility of variables.

Part D – Function Overloading *(where supported)*

Implement multiple functions having the same name but different parameter lists to perform different operations.

For example:

- Display Student Details
- Display Student Details with Academic Information
- Display Student Details with Academic Information and Contact Details

Observe the compiler's behaviour.

Note: Function Overloading is supported only in languages that provide compile-time function polymorphism. Where function overloading is not supported, implement equivalent functionality using distinct function names while preserving the same application logic.

Practical 8.3
Industry Scenario

A software development organization is evaluating different approaches to solve repetitive computational problems. The technical team wants to compare **iterative** and **recursive** implementations to understand their behavior, execution flow, readability, and efficiency before selecting an appropriate approach for software development.

As a software developer, develop a **Recursive Mathematical Operations Analyzer** to compare iterative and recursive solutions for commonly used mathematical computations.

Tasks to be Performed

Develop a **Recursive Mathematical Operations Analyzer** that performs the following operations.

Part A – Factorial

Implement factorial using

- Iterative approach
- Recursive approach

Display

- Factorial Value

Part B – Fibonacci Series
Generate Fibonacci Series using

- Iterative approach
- Recursive approach

Display the generated series.

Part C – Comparison
Compare both implementations with respect to

- Number of Function Calls
- Program Readability
- Code Complexity
- Execution Flow

Practical 8.4
Industry Scenario

A software company is developing a utility application to automate various numerical and data-processing tasks. The development team wants to solve repetitive problems by breaking them into smaller sub-problems using recursion instead of iterative constructs. This approach helps developers understand recursive traversal, decomposition, and problem-solving techniques for building efficient software solutions.

As a software developer, develop a **Recursive Utility Analyzer** to perform various recursive operations on numbers and arrays.

Tasks to be Performed

Develop a **Recursive Utility Analyzer** that performs the following operations.

Part A – Recursive Traversal

- Accept a positive integer **N**.
- Display numbers from **1 to N** using recursion.
- Display numbers from **N to 1** using recursion.

Part B – Recursive Number Processing

- Accept a positive integer.
- Reverse the entered number using recursion.
- Calculate the sum of digits of the entered number using recursion.

Part C – Recursive Number Conversion

- Accept a decimal number.
- Convert the decimal number into its binary representation using recursion.
- Display the generated binary number.

Part D – Comparative Analysis

. Compare the recursive implementations with iterative implementations (where applicable) and record observations regarding:

- Program readability
- Execution flow
- Number of function calls

- Ease of implementation

Questions for Discussion / Analysis / Interpretation / Viva

1. Explain the advantages of modular programming over monolithic programming.
2. Differentiate between user-defined functions and built-in functions with suitable examples.
3. Differentiate between function declaration, function definition, and function calling.
4. Explain the four categories of user-defined functions based on parameters and return values.
5. Differentiate between **Call by Value** and **Call by Reference** with suitable examples.
6. Explain the scope and visibility of local and global variables.
7. What is function overloading? Explain its advantages and limitations.
8. What is recursion? Why is a base condition mandatory in every recursive function?
9. Differentiate between recursion and iteration with suitable examples.
10. Explain recursive traversal and recursive decomposition with suitable applications.

Supplementary Problems

Level 1 – Easy

Develop an application to calculate the **sum of first N natural numbers** using recursion.

Level 2 – Medium

Develop an application to calculate the **power of a number (x^n)** using recursion.

Level 2 – Medium

Develop an application to **find the maximum and minimum values** among multiple numbers using separate user-defined functions.

Level 3 – Hard

Develop an application to **perform recursive binary search** on a sorted array.

Key Skills to be Addressed

- Modular programming
- User-defined functions
- Built-in functions
- Function declaration, definition, and calling
- Function parameter passing
- Call by Value
- Call by Reference
- Scope and visibility of variables
- Function overloading *(where supported)*
- Recursive programming
- Recursive traversal

	• Recursive decomposition • Comparative analysis of recursion and iteration **Learning Outcome** Upon successful completion of this practical set, students will be able to develop modular applications using user-defined and built-in functions, implement different function designs with appropriate parameter passing mechanisms, analyse variable scope and visibility, apply recursion to solve computational problems, compare recursive and iterative approaches, and develop structured, reusable, and maintainable software solutions. **Software / Tools / Platforms to be Used** • Visual Studio Code • Code::Blocks • GCC Compiler **Total Hours of Engagement** **6 Hours** • **120 Minutes** – Modular Student Record Management System using User-defined Functions • **120 Minutes** – Function Design, Parameter Passing, and Scope Analysis • **60 Minutes** – Recursive Mathematical Operations • **60 Minutes** – Recursive Traversal and Number Utility		
9	**Data Organization and Memory Management** **Aim** To understand address-based data access, pointer arithmetic, memory organization, multi-level referencing, and dynamic memory management for developing efficient and memory-safe applications. **Practical 9.1** **Industry Scenario** A software testing company is developing a **Memory Inspection Utility** to help trainee software engineers understand how data is stored and accessed in computer memory. Before working with complex software systems, developers must understand the relationship between variables, memory addresses, and indirect data access. As a software developer, develop a **Memory Address Explorer** to visualize memory locations and access data through addresses. **Tasks to be Performed** Develop a **Memory Address Explorer** that performs the following operations.	6	CO5

Part A – Address and Value

Declare variables of different data types.
Display:

- Variable Name
- Stored Value

Memory Address

Part B – Indirect Referencing

- Create address variables referring to the declared variables.
- Access and display values using indirect referencing.
- Modify values through indirect referencing.
- Observe the updated values.

Part C – Sequential Data Collections

- Declare a one-dimensional array.
- Display array elements using:
    1. Array indexing
    2. Address-based access
- Compare both approaches.

Expected Output

```
*********************************************
       MEMORY ADDRESS EXPLORER
*********************************************

---------- Variable Information ----------

Integer Value      : 25

Memory Address     : 0x61FF08

-------------------------------------------

Character Value    : A

Memory Address     : 0x61FF04

-------------------------------------------

Floating Value     : 85.60

Memory Address     : 0x61FF00

-------------------------------------------

Indirect Referencing

Stored Value       : 25
```

Modified Value : 30

Sequential Data Collection

Array using Index

10 20 30 40 50

Array using Address-based Access

10 20 30 40 50

Observation

Both approaches access the same memory locations.

Practical 9.2
Industry Scenario

The Student Record Management System developed in previous practicals stores student records using fixed-size storage. As the number of student admissions varies every academic session, the software development team has decided to redesign the application using dynamic memory allocation. The enhanced system should access records using address arithmetic, process records through address-based function calls, and safely release allocated memory after completing all operations.

As a software developer, enhance the **Student Record Management System** by implementing dynamic memory allocation, offset-based memory access, address-based function access, and safe memory deallocation.

Tasks to be Performed

Enhance the **Student Record Management System** by implementing dynamic memory management.

Part A – Dynamic Memory Allocation

- Accept the number of students to be registered.
- Dynamically allocate memory to store student records.
- Accept the following details for each student:
    1. Enrollment Number
    2. Student Name
    3. Branch
    4. Semester
    5. Percentage

6. Grade

Part B – Offset-based Memory Access

- Display all student records using:
- Address arithmetic (offset-based traversal)
- Display the memory address of each student record.
- Compare address-based traversal with sequential data collection access.

Part C – Address-based Function Access

- Develop user-defined functions that receive the address of the student data collection.
- Perform the following operations using address-based function access:
    1. Display all student records.
    2. Calculate the class average percentage.
    3. Display the highest percentage.
    4. Display the lowest percentage.

Part D – Dynamic Memory Deallocation

- Release the dynamically allocated memory after all operations are completed.
- Reset the address variable.
- Display an appropriate message confirming successful memory deallocation.

Part E – Observation

- Observe and compare:
    1. Dynamic memory allocation
    2. Offset-based memory access
    3. Address-based function access
    4. Memory deallocation
    5. Memory safety after resetting the address variable
    6. Record the observations.

Note: Automatic garbage collection is available in some programming languages. In this practical, memory is managed manually; therefore, dynamically allocated memory should always be released explicitly after program execution.

Expected Output

```
*****************************************
   STUDENT RECORD MANAGEMENT SYSTEM
*****************************************

Enter Number of Students : 3

Dynamic Memory Allocated Successfully

----------------------------------------

Enter Student Details
```

Student 1

Enrollment Number : 24CE001

Student Name : Amit Patel

Branch : CE

Semester : 1

Percentage : 86.40

Grade : A+

Student 2

Enrollment Number : 24CE002

Student Name : Riya Shah

Branch : IT

Semester : 1

Percentage : 82.60

Grade : A

Student 3

Enrollment Number : 24CE003

Student Name : Karan Mehta

Branch : CSE

Semester : 1

Percentage : 91.20

Grade : O

Student Records using Address-based Access

Address : 0x61FF00

24CE001 Amit Patel CE 1 86.40 A+

Address : 0x61FF48

24CE002 Riya Shah IT 1 82.60 A

Address : 0x61FF90

24CE003 Karan Mehta CSE 1 91.20 O

Academic Summary

Class Average Percentage : 86.73 %

Highest Percentage : 91.20 %

Lowest Percentage : 82.60 %

Dynamic Memory Released Successfully

Address Variable Reset Successfully

Program Executed Successfully

Practical 9.3
Industry Scenario

A software company is developing a **Memory Organization Analyzer** to help software developers understand how programs utilize different memory regions during execution. Before developing large-scale software applications, programmers must understand stack memory, heap memory, multi-level referencing, and common memory-related issues that affect software reliability.

As a software developer, develop a **Memory Organization Analyzer** to visualize different memory regions, demonstrate multi-level referencing, and identify unsafe memory practices.

Tasks to be Performed
Develop a **Memory Organization Analyzer** that performs the following operations.

Part A – Stack and Heap Memory

- Declare local variables inside a user-defined function.
- Dynamically allocate memory for variables.

- Display:
    1. Value
    2. Memory Address
- Identify which variables are stored in stack memory and which are stored in dynamically allocated memory.

Part B – Multi-level Referencing

- Declare
- Single-level address variable
- Double-level address variable
- Triple-level address variable *(up to three levels as prescribed in the syllabus)*
- Display
    1. Value stored
    2. Address stored at each level
    3. Final accessed value

Observe how data is accessed through multiple levels of referencing.

Part C – Memory Safety

- Demonstrate the following memory situations:
- Null Reference
- Invalid Reference *(after releasing dynamically allocated memory)*
- Uninitialized Address Variable *(for observation only)*
- Display appropriate messages indicating whether the reference is safe or unsafe.

Note: Unsafe memory situations should be demonstrated only for educational purposes. Avoid dereferencing invalid or uninitialized address variables.

Part D – Observation

. Compare the following concepts:

- Stack Memory Vs Dynamically Allocated Memory
- Single-level Vs Multi-level Referencing
- Safe Vs Unsafe Memory References
- Record the observations.

Expected Output

```
**********************************************
    MEMORY ORGANIZATION ANALYZER
**********************************************

---------- Stack Memory ----------

Local Variable Value

100

Memory Address
```

0x61FF08

---------- Dynamically Allocated Memory ----------

Allocated Value

250

Memory Address

0x0034F5A0

---------- Multi-level Referencing ----------

Original Value

50

Single-level Address

0x61FF20

Double-level Address

0x61FF18

Triple-level Address

0x61FF10

Value Accessed through Three Levels

50

---------- Memory Safety ----------

Null Reference

Safe (Reference is NULL)

Invalid Reference

Memory Released Successfully

Reference Invalid

Uninitialized Address Variable

Unsafe to Access

Questions for Discussion / Analysis / Interpretation to be Evaluated During/After Implementation

1. Differentiate between a variable's value and its memory address with suitable examples.
2. Explain the concept of indirect referencing and its significance in programming.
3. Differentiate between sequential data access and address-based data access.
4. Explain offset-based memory access arithmetic with a suitable example.
5. How does passing addresses to functions improve program efficiency?
6. Differentiate between stack memory and heap memory.
7. Explain the concept of multi-level referencing with suitable examples.
8. What are null references, invalid references, and uninitialized address variables? Why should they be avoided?
9. Explain the importance of dynamic memory allocation and deallocation in software development.
10. Why is it considered good programming practice to reset an address variable after releasing dynamically allocated memory?
11. Compare static memory allocation and dynamic memory allocation with suitable examples.
12. Explain the concept of memory safety and discuss common practices for developing memory-safe applications.

Supplementary Problems

Level 1 – Easy
Develop an application to display the values and memory addresses of all elements in a one-dimensional array using address-based access.

Level 2 – Medium
Develop an application to exchange the values of two variables using indirect referencing and display the updated values.

Level 2 – Medium
Develop an application to dynamically allocate memory for storing employee details and display all records using address-based traversal.

Level 3 – Hard

Develop an application to dynamically allocate memory for a two-dimensional matrix, perform matrix addition, display the result, and safely release all allocated memory.

Key Skills to be Addressed

- Address-based data access
- Indirect referencing
- Offset-based memory access arithmetic
- Address-based function access
- Dynamic memory allocation
- Dynamic memory deallocation
- Memory organization
- Stack and heap memory concepts
- Multi-level referencing
- Memory safety practices
- Efficient memory management

Learning Outcome

Upon successful completion of this practical set, students will be able to understand the relationship between values and memory addresses, implement address-based data access and offset-based memory traversal, process data using address-based function access, differentiate between stack and heap memory, apply multi-level referencing techniques, manage dynamically allocated memory safely, and develop efficient applications using sound memory management practices.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler

Total Hours of Engagement

6 Hours

- **120 Minutes** – Memory Address Explorer
- **120 Minutes** – Dynamic Student Record Management System using Address-based Access
- **120 Minutes** – Memory Organization Analyzer

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of address-based data access, pointer arithmetic, dynamic memory allocation, and memory deallocation	**40**
Understanding of memory organization, address-based function access, and memory safety concepts	**25**
Program correctness, testing, and output formatting	**20**
Code readability, documentation, and viva	**15**

	performance		
10	**Data Modeling** **Aim** To model real-world entities using composite data types, organize related information efficiently, understand shared memory composite data representation, analyze padding and alignment, and evaluate different composite data representations for efficient software design. **Practical 10.1** **Industry Scenario** The existing **Student Record Management System** stores student information using multiple independent variables and arrays, making the application difficult to maintain as additional information such as contact details and academic information are introduced. The software development team has decided to redesign the application using **Composite Data Types (Structures)** to organize related information into meaningful entities and improve software maintainability. As a software developer, redesign the **Student Record Management System** using **Composite Data Types (Structures)** and hierarchical data organization. **Tasks to be Performed** **Part A – Composite Data Modeling (Structure)** • Design a **Composite Data Type (Structure)** to represent a student. • Include the following information: 1. Enrollment Number 2. Student Name 3. Branch 4. Semester 5. Percentage 6. Grade **Part B – Hierarchical Composite Data Grouping (Nested Structures)** Extend the student model by grouping additional information into separate **Composite Data Types (Nested Structures)**. Example Student • Academic Information • Contact Information	6	CO6

- Academic Information
- Semester
- Percentage
- Grade

Contact Information

1. Mobile Number
2. Email ID

Part C – Sequential Collection of Composite Data Types (Array of Structures)

- Create a sequential collection of student records.
- Accept details of multiple students.
- Display all student records in a formatted report.

Part D – Address-based Data Sharing (Pointer to Structure)

Develop user-defined functions that receive the address of a student record.

- Perform the following operations.
- Display Student Details
- Update Contact Information
- Display Academic Information

Expected Output

```
*****************************************
   STUDENT RECORD MANAGEMENT SYSTEM
*****************************************

Enter Number of Students : 2

-----------------------------------------

Student 1

Enrollment Number : 24CE001

Student Name     : Amit Patel

Branch          : CE

Semester        : 1

Percentage      : 86.40

Grade           : A+

Mobile Number    : 9876543210

Email ID        : amit@gmail.com
```

```
----------------------------------------

Student 2

Enrollment Number : 24CE002

Student Name      : Riya Shah

Branch            : IT

Semester          : 1

Percentage        : 82.60

Grade             : A

Mobile Number     : 9876501234

Email ID          : riya@gmail.com

----------------------------------------

Student Records

24CE001  Amit Patel  CE

Semester : 1

Percentage : 86.40

Grade : A+

Mobile : 9876543210

Email : amit@gmail.com

----------------------------------------

24CE002  Riya Shah  IT

Semester : 1

Percentage : 82.60

Grade : A

Mobile : 9876501234

Email : riya@gmail.com

----------------------------------------

Program Executed Successfully

----------------------------------------
```

Practical 10.2
Industry Scenario

A software company is developing applications for memory-constrained embedded systems. Before selecting an appropriate composite representation, the development team wants to analyze memory utilization, data organization, and storage efficiency using different composite data representations.

As a software developer, develop a **Composite Data Representation Analyzer** to study **Composite Data Types (Structures)**, **Shared Memory Composite Data Representation (Union)**, memory padding, alignment, and their trade-offs.

Tasks to be Performed

Part A – Composite Data Representation (Structure)

Create a **Composite Data Type (Structure)** to represent product information.
Display:

- Size of each data member
- Total size of the structure

Part B – Padding and Alignment (Structure)

- Create two **Structures** containing identical data members but arranged in different orders.
- Display the total size of each structure.
- Compare the effect of padding and alignment on memory utilization.

Part C – Shared Memory Composite Data Representation (Union)
Create a **Composite Data Type (Union)** containing:

- Integer
- Floating-point Number
- Character

Store values one after another in each member.
Observe how updating one member affects the values of the remaining members.

Part D – Trade-off Analysis (Structure vs Union)

Compare **Structures** and **Unions** with respect to:

- Memory Utilization

- Data Organization
- Simultaneous Data Storage
- Shared Memory
- Suitable Applications

Record your observations.

Expected Output

```
*****************************************
 COMPOSITE DATA REPRESENTATION ANALYZER
*****************************************

Composite Data Type A

Size : 24 Bytes

-----------------------------------------

Composite Data Type B

Size : 16 Bytes

-----------------------------------------

Padding and Alignment Observed

-----------------------------------------

Shared Memory Representation

Integer Value : 120

Floating Value : 120.00

Character Value : x

After Updating Floating Value

Integer Value Changed

Character Value Changed

-----------------------------------------

Trade-off Analysis

Structure

Separate Memory for Every Member

Suitable for Storing Complete Records

-----------------------------------------
```

Union

Shared Memory for All Members

Suitable for Memory-efficient Applications

Program Executed Successfully

Questions for Discussion / Analysis / Interpretation to be Evaluated During/After Implementation

- What is a Composite Data Type? Explain its advantages over primitive data types.
- Differentiate between primitive data types and composite data types with suitable examples.
- Explain how **Composite Data Types (Structures)** help in modeling real-world entities.
- What is **Hierarchical Composite Data Grouping (Nested Structures)**? Explain with a suitable example.
- Differentiate between passing a composite data type by value and passing it by address.
- What is **Shared Memory Composite Data Representation (Union)**? How does it differ from a structure?
- Explain the concepts of **padding** and **alignment**. Why are they important in memory organization?
- How does the arrangement of members in a structure affect memory utilization?
- Compare **Structure** and **Union** with respect to memory allocation, data storage, and suitable applications.
- Why is trade-off analysis important while selecting an appropriate composite data representation?
- Explain how composite data modeling improves software maintainability and scalability.
- Identify suitable real-world applications where structures and unions can be effectively used.

Supplementary Problems
Level 1 – Easy
Develop an application to maintain **employee information** using **Composite Data Types (Structure)** and display the records in a formatted report.

Level 2 – Medium
Develop an application to maintain **library book information** using **Nested Structures** for book details and publisher information.

Level 2 – Medium

Develop an application to compare the memory occupied by two **Structures** containing the same members arranged in different orders and record the observations.

Level 3 – Hard
Develop an application to compare **Structure** and **Union** by storing integer, floating-point, and character values, analyze memory utilization using sizeof(), and prepare a comparative report highlighting their suitable applications.

Key Skills to be Addressed
- Composite data modeling
- Composite Data Types (Structures)
- Shared Memory Composite Data Representation (Unions)
- Hierarchical Composite Data Grouping (Nested Structures)
- Sequential collection of composite data types
- Address-based reference to composite data types
- Address-based data sharing
- Memory organization
- Padding and alignment
- Trade-off analysis between composite representations
- Real-world data modeling

Learning Outcome
Upon successful completion of this practical set, students will be able to model real-world entities using **Composite Data Types (Structures)**, organize related information through hierarchical grouping, implement sequential collections of composite data, access and manipulate composite data using address-based references, analyze **Shared Memory Composite Data Representation (Unions)**, evaluate the effects of padding and alignment on memory utilization, and select an appropriate composite representation based on application requirements.

Software / Tools / Platforms to be Used
- Visual Studio Code
- Code::Blocks
- GCC Compiler

Total Hours of Engagement
6 Hours

- **4 Hours** – Composite Data Modeling using Student Record Management System *(Composite Data Types – Structure)*
- **2 Minutes** – Composite Data Representation Analyzer *(Shared Memory Composite Data Representation – Union, Padding, Alignment, and Trade-off Analysis)*

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)

Correct implementation of Composite Data Types (Structures), hierarchical grouping, and address-based data sharing	**40**
Understanding of Shared Memory Composite Data Representation (Union), padding, alignment, and trade-off analysis	**25**
Program correctness, testing, and output formatting	**20**
Code readability, documentation, and viva performance	**15**

11	**Practical Set – 11** **Advanced Concepts** **Aim** To understand program execution, compile-time and run-time memory management, common low-level programming errors, and implement appropriate error-handling techniques for developing reliable software applications. **Practical 11.1** **Industry Scenario** A software testing team is evaluating newly developed applications before deployment. During testing, programs may encounter invalid user inputs, arithmetic errors, memory-related issues, and file access failures. Before releasing the software, developers must identify these situations and implement appropriate error-handling mechanisms to ensure reliable program execution. As a software developer, develop a Program Execution and Error Handling Analyzer to demonstrate common execution- time errors and implement suitable error-handling techniques. **Tasks to be Performed** Develop a Program Execution and Error Handling Analyzer that performs the following operations. **Part A – Compile-time and Run-time** Memory Observation 1. Declare global and local variables. 2. Dynamically allocate memory for storing a numeric value. 3. Display: • Global variable value • Local variable value • Dynamically allocated value 4. Observe the difference between compile-time and run-time memory allocation. **Part B – Low-level Error** Demonstration Demonstrate the following situations safely: 5. Arithmetic Overflow • Perform an arithmetic operation that exceeds the storage capacity of the selected data type. • Observe the resulting output. 6. Pointer Aliasing • Create two address variables referring to the same memory location. • Modify the value using one address and observe the effect through the other.	6	CO7

7. Memory Leak (Conceptual Demonstration)
    - Allocate memory dynamically.
    - Observe the importance of releasing dynamically allocated memory after use.

Part C – Error Handling

Implement suitable validation for the following situations.

8. Accept two numbers and perform division.
9. Display an appropriate error message if:
    - Division by zero is attempted.
    - Invalid numerical input is entered.
10. Attempt to open a file that does not exist and display an appropriate error message.

Part D – Program Execution

Observation

11. Observe the program execution flow under:
    - Normal execution
    - Invalid input
    - Arithmetic error
    - File opening error
12. Record your observations regarding program reliability and error handling.

Expected Output

```
**********************************************
PROGRAM EXECUTION &amp;amp; ERROR HANDLING ANALYZER
**********************************************
Compile-time and Run-time Memory
Global Variable Value : 100
Local Variable Value : 200
Dynamic Memory Value : 300
------------------------------------------
Low-level Error Demonstration
Arithmetic Overflow Observed
------------------------------------------
Pointer Aliasing
Original Value : 50
Updated Value : 75
Both Address Variables Refer to the Same Memory Location
------------------------------------------
Memory Leak Demonstration
Dynamic Memory Allocated
Memory Released Successfully
------------------------------------------
```

Arithmetic Operation
Enter Dividend : 25
Enter Divisor : 0

Faculty of Technology, CHARUSAT CEUC103_CPF

Error : Division by Zero is not Allowed.

File Operation
Unable to Open File.

Program Executed Successfully

Practical 11.2

Industry Scenario

A publishing company maintains digital documents for reports and manuals. To ensure data safety and easy retrieval, the company stores information in text files, creates backup copies, and performs various file operations such as reading, writing, copying, and random access. Before integrating file handling into business applications, software developers must understand different file operations and the functions used to manage files efficiently.

As a software developer, develop a File Organization and File Operations Utility to perform various file operations using sequential file organization.

Tasks to be Performed

Develop a File Organization and File Operations Utility that performs the following operations.

Part A – File Creation and Writing

1. Create a text file named Document.txt.
2. Accept multiple lines of text from the user.
3. Store the entered contents into Document.txt.
4. Display an appropriate message after successful file creation and writing.

Part B – File Reading

5. Open Document.txt in read mode.
6. Read and display the complete contents of the file.
7. Detect the end of the file using the appropriate file handling function.

Part C – File Copying and Error Detection

8. Create another file named BackupDocument.txt.
9. Copy all contents from Document.txt to BackupDocument.txt.

10. Display a suitable error message if any file cannot be opened.

11. Verify that the backup file has been created successfully.

Part D – Random File Access

12. Display the current file position using the appropriate
file positioning function.

13. Move the file pointer to different locations using suitable file positioning functions.

14. Read and display the contents after repositioning the file pointer.

15. Reset the file pointer to the beginning of the file and display the complete contents again.

16. Record your observations regarding sequential file organization and random file access.

Expected Output

```
**********************************************
FILE ORGANIZATION &amp;amp; FILE OPERATIONS UTILITY
**********************************************
Document.txt Created Successfully
------------------------------------------
Enter File Contents
Programming Fundamentals Laboratory
Student Record Management System
File Handling Practical
------------------------------------------
Data Written Successfully
------------------------------------------
Reading Document.txt
 Programming Fundamentals Laboratory
Student Record Management System
File Handling Practical
------------------------------------------
End of File Reached Successfully
------------------------------------------
Creating BackupDocument.txt
Contents Copied Successfully
Backup File Created Successfully
------------------------------------------
Current File Position : 78
Moving File Pointer...
Displaying Contents after Repositioning
Student Record Management System
------------------------------------------
Resetting File Pointer
Programming Fundamentals Laboratory
```

Student Record Management System
File Handling Practical

Program Executed Successfully

Practical 11.3

Industry Scenario

The Student Record Management System developed in previous practicals stores student records only during program execution. Once the application is closed, all records are lost. To preserve academic information permanently, the institute has decided to implement Sequential File Organization. The enhanced system should create student record files, store records permanently, retrieve stored records, maintain backup copies, and handle file-related errors gracefully.

As a software developer, enhance the Student Record Management System by implementing Sequential File Organization, file operations, and error handling for persistent data management.

Tasks to be Performed

Enhance the Student Record Management System developed in previous practicals by implementing persistent file storage.

Part A – Create and Write Student Records

1. Create a file named StudentRecords.txt.
2. Accept the following student details:
    - Enrollment Number
    - Student Name
    - Branch
    - Semester
    - Percentage
    - Grade
3. Store all student records in StudentRecords.txt.
4. Display a confirmation message after successfully storing the records.

Part B – Read and Display Student Records

5. Open StudentRecords.txt in read mode.
6. Read and display all stored student records.
7. Detect the end of the file using the appropriate file handling function.
8. Display a suitable message after all records have been read successfully.

Part C – Backup Student Records

9. Create a backup file named BackupStudentRecords.txt.

10. Copy all student records from StudentRecords.txt to BackupStudentRecords.txt.

11. Display a confirmation message after successful backup.

Part D – Error Handling

12. Display an appropriate error message if:

- The student record file cannot be created.
- The student record file cannot be opened for reading.
- The backup file cannot be created.

13. Ensure that the program terminates gracefully whenever a file operation fails.

Part E – Observation

14. Observe and compare:

- Temporary data storage during program execution.
- Permanent storage using files.
- File creation, writing, reading, and backup operations.
- Importance of error handling in file processing.

Record your observations.

Expected Output

```
****************************************
STUDENT RECORD MANAGEMENT SYSTEM
****************************************
StudentRecords.txt Created Successfully
----------------------------------------
Enter Number of Students : 2
----------------------------------------
Student 1
Enrollment Number : 24CE001
Student Name : Amit Patel
Branch : CE
Semester : 1
Percentage : 86.40
Grade : A+
----------------------------------------
Student 2
Enrollment Number : 24CE002
Student Name : Riya Shah
Branch : IT
Semester : 1
```

Percentage : 82.60
Grade : A

Student Records Stored Successfully

Reading StudentRecords.txt
Enrollment : 24CE001
Name : Amit Patel
Branch : CE
Semester : 1
Percentage : 86.40
Grade : A+

Enrollment : 24CE002
Name : Riya Shah
Branch : IT
Semester : 1
Percentage : 82.60
Grade : A

End of File Reached Successfully

Creating BackupStudentRecords.txt
Backup Completed Successfully

Program Executed Successfully

Questions for Discussion / Analysis / Interpretation to be Evaluated During/After Implementation

1. Differentiate between compile-time memory management and run-time memory management.
2. Explain the causes and effects of memory leaks with suitable examples.
3. What is pointer aliasing? Explain its advantages and possible drawbacks.
4. Differentiate between arithmetic overflow and logical errors in a program.
5. Explain the importance of error handling in software development.
6. Differentiate between text files and binary files with suitable applications.
7. What is Sequential File Organization? Explain its advantages and limitations.
8. Explain the purpose of the following file handling functions with suitable examples:
    - feof()
    - ferror()
    - ftell()

- fseek()
- rewind()

9. Why should a program always verify whether a file has been opened successfully before performing file operations?
10. Explain how persistent storage improves the functionality of the Student Record Management System.
11. Differentiate between temporary data storage and persistent data storage.
12. Explain the role of error handling in developing reliable file-based applications.

Supplementary Problems

Level 1 – Easy

1. Develop an application to create a text file, store five city names, and display the contents of the file.

Level 2 – Medium

2. Develop an application to count the number of characters, words, and lines present in a text file.

Level 2 – Medium

3. Develop an application to copy the contents of one text file into another file and display an appropriate message if either file cannot be opened.

Level 3 – Hard

4. Develop an application to merge the contents of two text files into a third file and display the merged contents.
5. Implement appropriate error handling for all file operations.

Key Skills to be Addressed

- Compile-time and run-time memory management
- Identification of low-level programming errors
- Program execution analysis
- Error handling and exceptional handling
- Text file handling
- Sequential file organization
- File creation, reading, writing, and copying
- File positioning techniques
- End-of-file and file error detection
- Persistent data management
- Reliable software development

Learning Outcome

Upon successful completion of this practical set, students will be able to distinguish between compile-time and run-time memory management, identify common low-level

programming errors, implement appropriate error-handling techniques, perform text file operations using sequential file organization, utilize standard file handling functions for file positioning and error detection, and develop reliable applications with persistent data storage by enhancing the Student Record Management System.

Software / Tools / Platforms to be Used

- Visual Studio Code
- Code::Blocks
- GCC Compiler

Total Hours of Engagement

3 Hours

- **60 Minutes** – Program Execution and Error Handling Analyzer.
- **60 Minutes** – File Organization and File Operations Utility.
- **60 Minutes** – Persistent Student Record Management System.

Rubrics (Practical Evaluation / Viva)

Criteria	Weight (%)
Correct implementation of memory management concepts, error handling, and file operations	40
Implementation of persistent storage and integration into the Student Record Management System	25
Program correctness, testing, output formatting, and file management	20
Code readability, documentation, and viva performance	15